

Part 1:

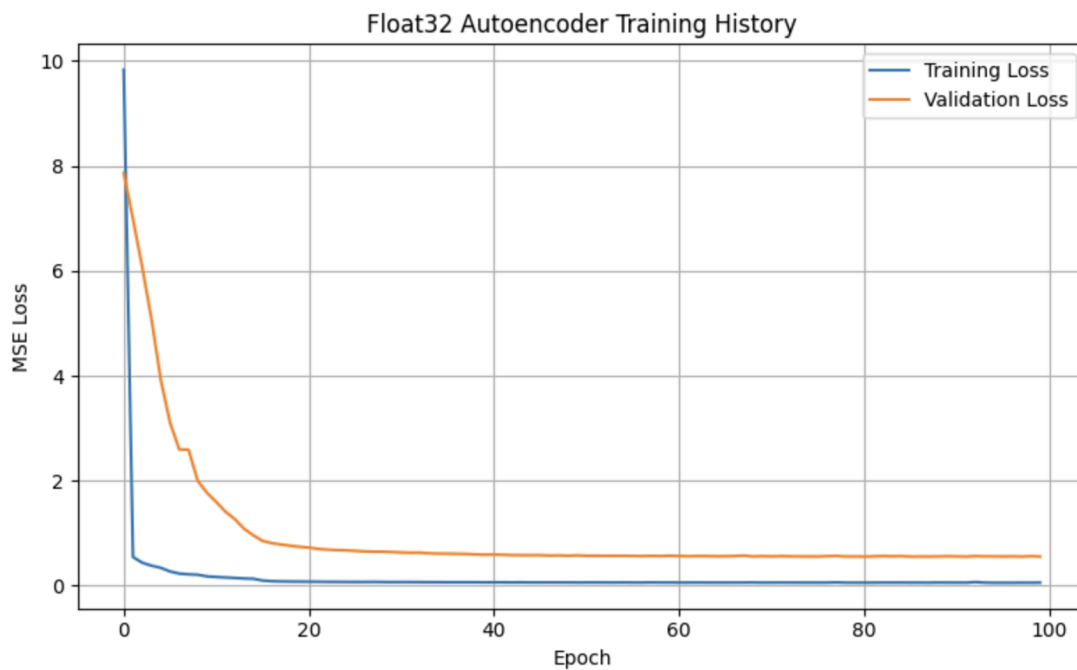
Edge Impulse Link: <https://studio.edgeimpulse.com/public/999037/live>

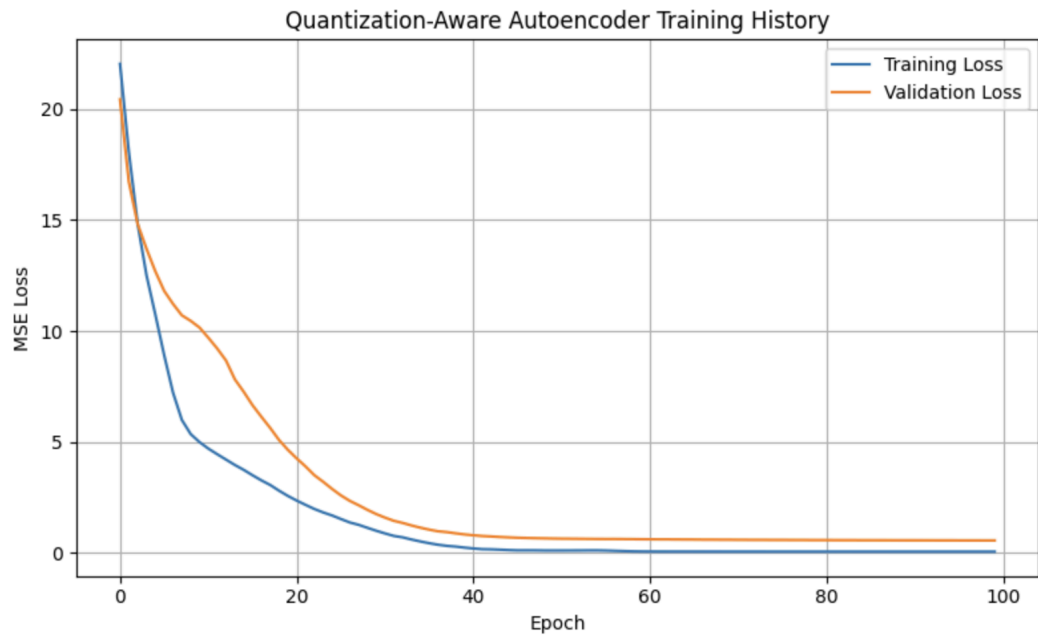
Edge Impulse Screenshots:

```
Predictions (DSP: 25 ms., Classification: 1 ms., Anomaly: 1 ms.):  
  nominal: 0.34375  
  off: 0.65625  
  anomaly score: 1000.000  
  
Starting inferencing in 2 seconds...  
Sampling...
```

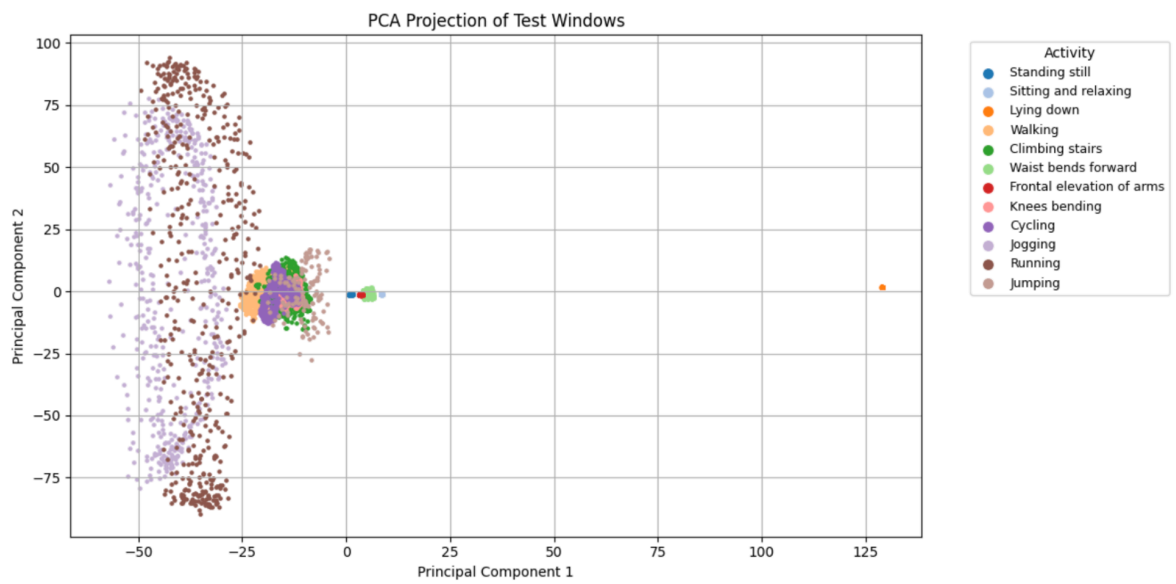
Section 2: Autoencoder Model Results:

Training and validation loss curves:

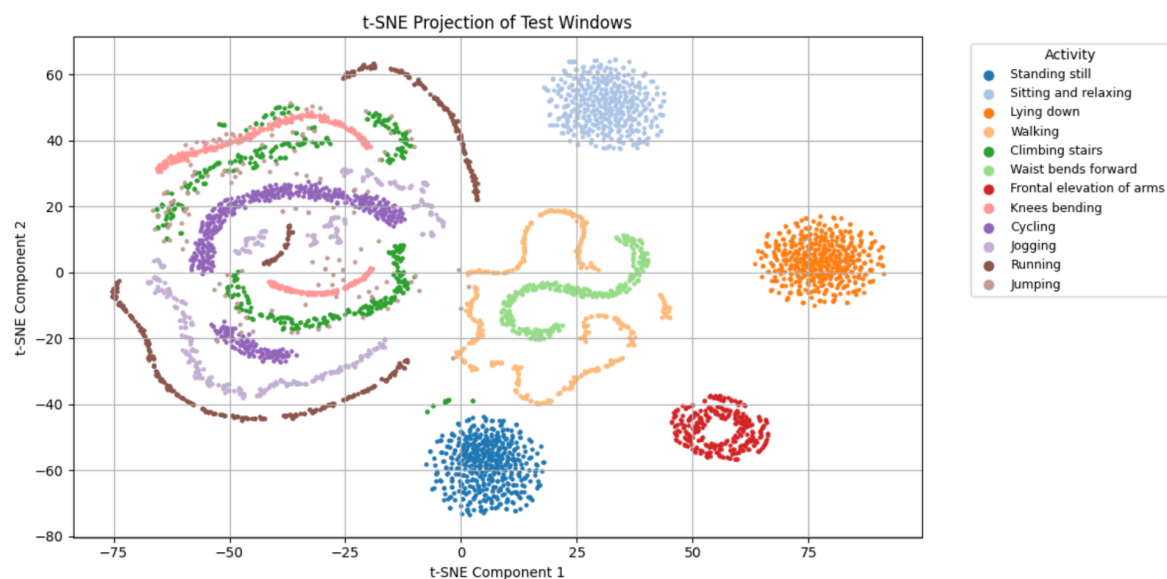




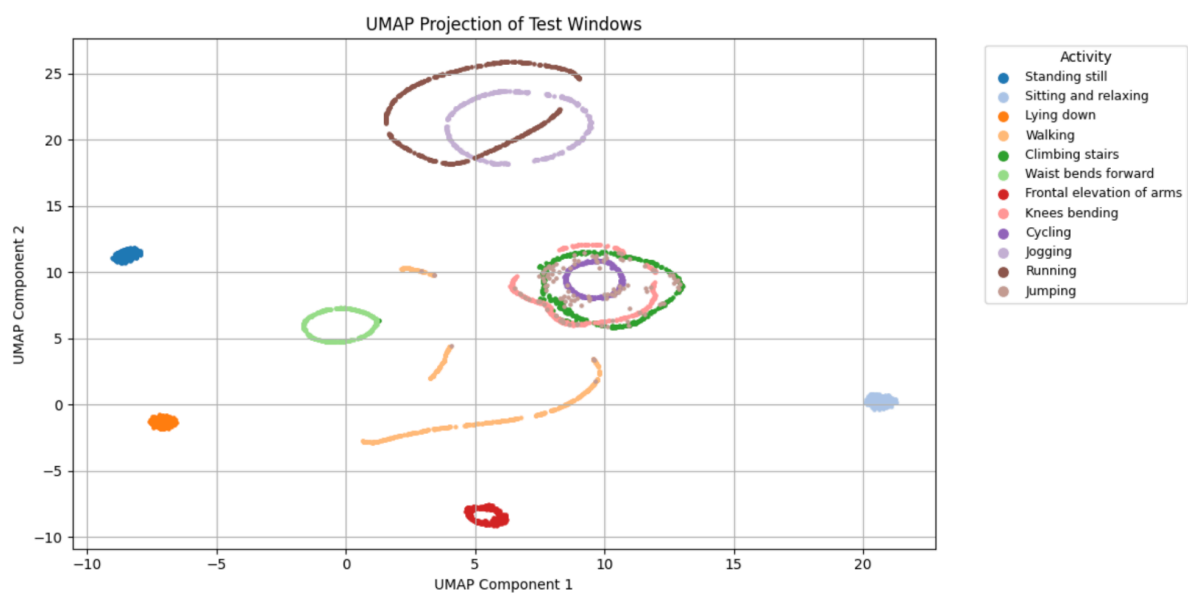
PCA visualization:



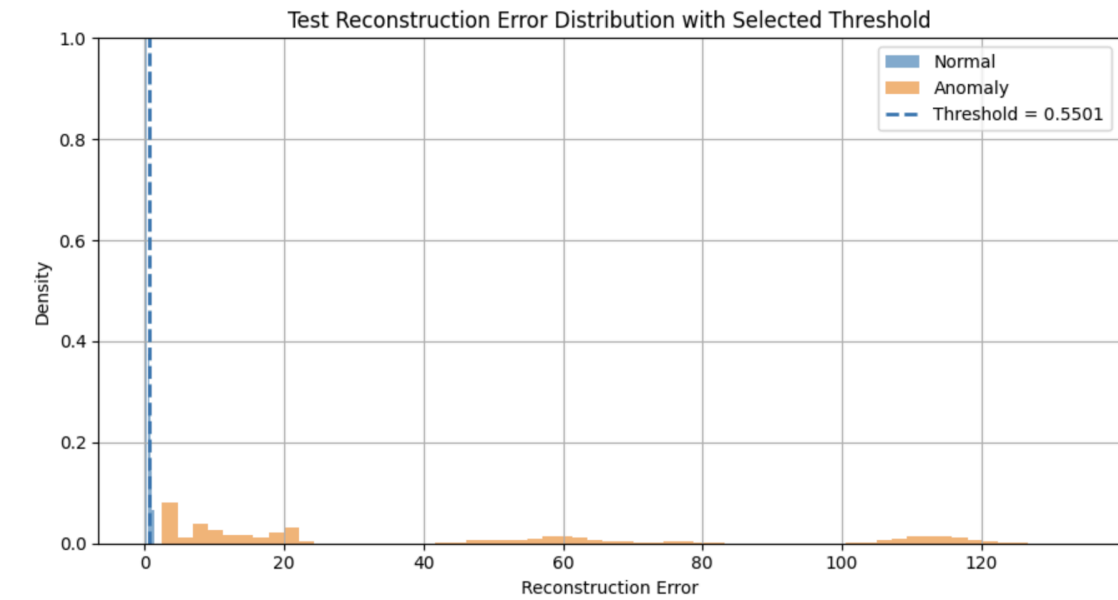
t-SNE visualization:



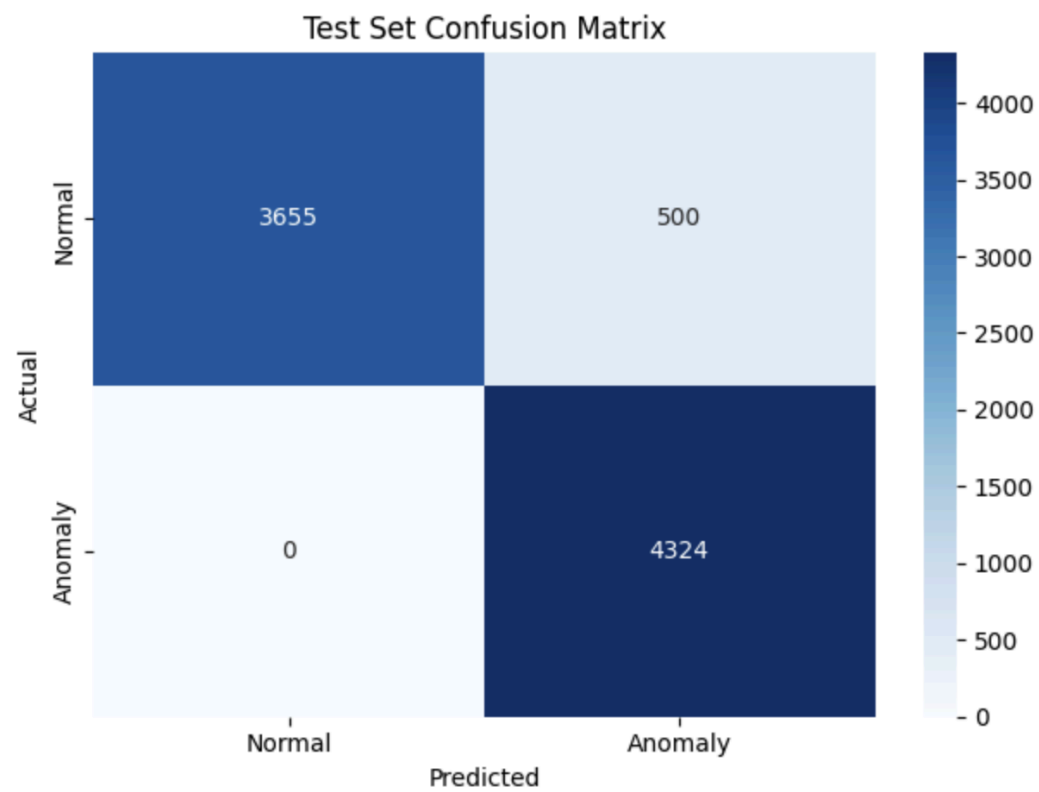
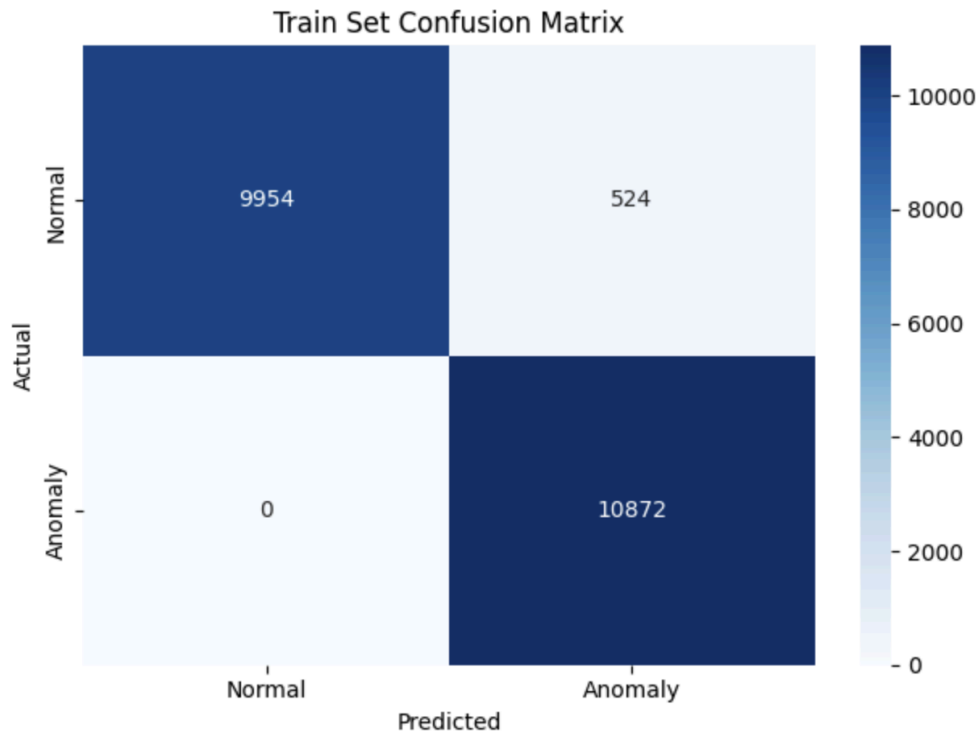
UMAP visualization:



Reconstruction Error Distribution:



Confusion Matrix:



Arduino Screenshot:

```
Result: Normal activity
Reconstruction error: 0.572503
Result: Normal activity
Reconstruction error: 0.573945
Result: Normal activity
Reconstruction error: 0.563937
Result: Normal activity
```

Discussion:

Part 1: Edge Impulse Results

- The raw inputs that the edge impulse project used was the accX, accY, and accZ. These were used to obtain the collect motion windows which were then used for the classification and anomaly detection parts.
- The NN classifier used the spectral features that were obtained from the raw accelerometer signals. There were a total of 33 input features from the axes. For example there was accX RMS, accX Peak 1 Freq, accX Peak 1 Height, accY RMS, accZ RMS, accZ Spectral 2-5, and accZ Spectral power 2-5.
- The outputs were off and nominal.

The deployment results essentially show that the Arduino board had the ability to collect the accelerometer data, process the classifier, and also output the anomaly score/ classes. One thing to keep in mind, however, is that if there's a higher anomaly score, then this may mean that the input is outside the regular distribution of data that the model was trained on. This is true even if there's a confident prediction (nominal, off). In conclusion, the results should be cross-referenced with the anomaly score.

Part 2: Autoencoder Model Results

The threshold value that I used for my anomaly detection was 0.5501. I chose this value because it was the 95th percentile of the reconstruction errors. This allowed for the high error windows to be categorized as anomalies while the normal windows are normal.

The reconstruction error helps discriminate between normal and anomalous activity because it measures how different the output is from the original window. When the model gets anomalies, the signal cannot be constructed correctly, which makes the error increase.

The Arduino deployment needs to preserve the trained int8 model, the threshold for the reconstruction error, and order of the features, and the size of the input. The model needs to get 100 samples, and the quantization parameters are also important.

The arduino sketch must use the same assumptions as the notebook because the model was trained with specificity. There is a specific shape, specific axes, and scaling methods. If there was a difference, the input data will not match the distribution.

The main limitations are the memory, compute power, noise from the sensors, and threshold sensitivity. For example, if the value of the threshold is too low, fp's can be created, whereas a high threshold value causes anomalies to be missed. Additionally, the model needs to be efficient because the arduino is not that powerful.